

Ankara Yıldırım Beyazıt University
Department of Computer Engineering

Cloud Computing Course Project

PricePulse

A Twelve-Factor E-Commerce Price Monitoring Platform

Author: Sarper AVCI

Submission: 22 May 2026

Repository: <https://github.com/sarperfiles/pricepulse>

Abstract

This report describes the design, implementation, and deployment of PricePulse, a cloud-based Software-as-a-Service (SaaS) platform that monitors e-commerce product prices and notifies users of changes. The system was developed in conformance with the twelve-factor application methodology proposed by Wiggins (2017), with each factor mapped to a concrete architectural or operational choice. The implementation uses Python 3.12 with the FastAPI asynchronous web framework on the server side, React with TypeScript on the client side, PostgreSQL for persistent storage, Redis for caching and message brokering, and a stealth-augmented variant of Playwright for scraping JavaScript-heavy sites. The platform is packaged as a set of stateless container images, orchestrated locally with Docker Compose and portable to Kubernetes, Amazon Elastic Container Service (ECS), or Google Cloud Run without code modification. A continuous integration pipeline implemented in GitHub Actions performs linting, unit testing, and image publication on every merge. End-to-end functionality was verified against four real e-commerce vendors: Amazon, eBay, GetYourGuide, and Hepsiburada.

1 Introduction

The expansion of online retail has produced a market in which prices fluctuate at minute-level granularity. Manual price monitoring does not scale; existing commercial monitoring services are either restricted to a single vendor, rate-limited, or closed to programmatic integration. A further difficulty arises from the prevalence of client-side rendering: prices on modern e-commerce pages are frequently injected by JavaScript after the initial HTTP response, defeating naive HTML scrapers, and anti-bot systems such as Cloudflare and DataDome routinely block unstealthed automation tools.

PricePulse addresses these constraints with a platform-agnostic scraping engine, a programmatic REST application programming interface (API), and a single-page web dashboard. Users submit a product Uniform Resource Locator (URL); the system identifies the platform, schedules periodic scrapes, persists the full price history, evaluates user-defined alert rules, and surfaces in-application notifications when those rules are triggered.

The remainder of this report is organised as follows. Section 2 summarises the twelve-factor methodology that guided the design and shows how each factor was satisfied. Section 3 describes the system architecture. Section 4 details the implementation, including the scraping engine that distinguishes the project technically. Section 5 addresses cloud deployment and migration paths. Section 6 reports on testing and verification. Section 7 discusses challenges encountered and the resolutions adopted. Section 8 concludes.

2 Twelve-Factor Conformance

The twelve-factor methodology specifies a set of disciplines for constructing portable, scalable SaaS applications. PricePulse satisfies each of the twelve factors as summarised in Table 1.

3 System Architecture

PricePulse is structured as a modular monolith augmented by a separate worker process. The two processes share a single source tree and a common data layer but execute independently.

3.1 Process Topology

The API process, implemented with FastAPI on top of Uvicorn, accepts all HTTP traffic. It performs authentication, business validation, persistence through the asynchronous SQLAlchemy

2.0 Object–Relational Mapper (ORM), and the enqueueing of background jobs onto the Redis queue. The worker process, implemented with the Async Redis Queue library, consumes those jobs, executes scrapes, and writes results back to PostgreSQL.

This split was adopted in preference to two alternatives. A pure monolith was rejected because long-running browser-based scrapes would block the API event loop and introduce tail-latency spikes. A full microservices decomposition was rejected as premature for a single-tenant deployment, as it would impose operational overhead (service discovery, distributed tracing, deployment orchestration) without delivering proportional scalability benefit. The hybrid preserves a single source of truth for SQLAlchemy models and Alembic migration history while isolating the failure domain of the scraper from the user-facing API.

3.2 Data Model

Six entities form the persistence layer: `users`, `products`, `price_history`, `alert_rules`, `notifications`, and `scrape_jobs`. Foreign-key relationships and indexes are summarised in Table 2.

3.3 Data Flow

A typical interaction proceeds as follows. The user submits a product URL via the dashboard. The API auto-detects the platform from the URL domain, stores the `Product` record, and enqueues an immediate scrape job. The worker, polling the Redis queue, dequeues the job within seconds and selects a scraping strategy: a stealthed Chromium browser for JavaScript-heavy platforms (GetYourGuide, Hepsiburada, Booking, Airbnb, Expedia), or a static HTTP client with HyperText Markup Language (HTML) parsing for the remainder. The extracted price is persisted to `price_history`; the parent `Product` row’s `current_price` and `next_scrape_at` fields are updated. Active alert rules are evaluated, and any triggered rule produces a `notifications` row. A periodic cron task within ARQ re-checks the database every minute for products whose `next_scrape_at` has elapsed and enqueues the next round.

4 Implementation

4.1 Technology Stack

The components and the rationale for their selection are summarised in Table 3. Where the assignment specification listed example technologies, the relevant equivalent in PricePulse is indicated.

The frontend is implemented in React 19 with TypeScript on Vite. Styling uses Tailwind CSS version 4. Charting is provided by Recharts. The compiled production bundle ships in a separate container and is served either by the Vite preview server or by any static file host.

4.2 Authentication

Authentication accepts two credential types. JSON Web Tokens (JWT) are issued on email-and-password login and used for browser sessions; a refresh token, valid for seven days, allows access-token renewal. API keys are issued on user creation and accepted via the `X-API-Key` HTTP header for programmatic access. Passwords are hashed using `bcrypt` directly. An earlier reliance on `passlib[bcrypt]` was removed when its compatibility with recent `bcrypt` releases regressed under Python 3.12.

4.3 Scraping Engine

The scraping subsystem is the technically distinguishing element of the project. It accommodates three vendor categories: statically rendered pages where prices appear in the HTTP response body; JavaScript-rendered pages where prices are injected client-side; and bot-protected pages that respond to naive HTTP clients with status 403 or a CAPTCHA challenge.

The selection logic, executed once per scrape job, inspects the product’s `platform` field. JS-heavy platforms route to a Patchright-driven Chromium instance. Patchright is a maintained fork of Playwright that patches the `Runtime.enable` Chrome DevTools Protocol leak relied upon by Cloudflare, DataDome, and Akamai for headless-browser detection. BrowserForge generates a realistic Chrome fingerprint, including User-Agent and `sec-ch-ua-*` client hints, on each session. All other platforms route to an asynchronous `httpx` client paired with BeautifulSoup version 4 for HTML parsing. If the static client returns `not_found` for a product whose platform is unspecified, the task retries the same URL with the stealth browser as a fallback.

Each platform exposes an extractor object that attempts a chain of strategies until one yields a price. The chain is: parsing of `schema.org/Product` JavaScript Object Notation for Linked Data (JSON-LD); inspection of the Next.js `__NEXT_DATA__` hydration payload; reading of Open Graph price meta tags; matching of hand-tuned Cascading Style Sheets (CSS) selectors specific to the platform; and a regular-expression fallback that scans the entire response for currency-prefixed numeric patterns and returns the modal price found. This layered approach tolerates many vendor-side HTML changes, since a single broken strategy rarely invalidates all five.

The system additionally extracts the product title from the rendered page (JSON-LD `name`, then Open Graph `title`, then the first `<h1>`) and uses it to replace any placeholder name generated from the URL domain on the first successful scrape. This spares the user from having to type a product name when adding a URL.

End-to-end verification was performed against four vendors. The results, including the scraper variant invoked, are reported in Table 4.

4.4 Frontend

The frontend is a single-page application providing seven views: authentication, dashboard, products list, product detail, notification feed, account settings, and a fall-through 404 route. Tabular product listings support sorting, filtering, pagination, bulk deletion, a table-card view toggle, and per-row pause and resume of tracking. Product detail incorporates a Recharts-rendered price-history line chart with selectable time ranges, a tabular history view, exportable to Comma-Separated Values (CSV), and an alert-rule subform. User-facing operational concerns—toast notifications, loading skeletons, confirmation dialogues, keyboard shortcuts, and a mobile-responsive sidebar—are implemented uniformly across all views.

5 Cloud Deployment

The complete development stack starts with a single command:

```
docker compose up -d
docker compose exec api alembic -c backend/alembic.ini upgrade head
```

The Docker Compose file declares five services: PostgreSQL 16 Alpine, Redis 7 Alpine, the API container, the worker container, and the Vite development server for the frontend. PostgreSQL and Redis include healthchecks; the API and worker enter the dependency graph only once those healthchecks pass. For external testing, the stack was provisioned on a virtual private server fronted by a Cloudflare reverse proxy, which terminates Transport Layer Security (TLS) and

masks the origin address. The compose file binds the frontend, API, PostgreSQL, and Redis to the high local ports 50173, 50344, 50432, and 50379 respectively to avoid collisions with other services on the host.

Migration to a managed cloud platform is straightforward in each of the three commonly used cases. For Amazon Elastic Container Service, the existing images are pushed to Amazon Elastic Container Registry, the Compose definition is translated to an ECS task definition, `DATABASE_URL` is pointed at an RDS PostgreSQL instance, and `REDIS_URL` is pointed at an ElastiCache Redis instance. For Google Cloud Run, the API container is registered as a Cloud Run service, with Cloud SQL for PostgreSQL and Memorystore for Redis as the backing services. For self-managed Kubernetes, each container maps to a standard Deployment with an associated Service and ConfigMap; the modular-monolith-plus-worker split aligns naturally with two separate Deployments scaling independently. None of these migrations requires a code change.

6 Testing and Verification

The backend test suite, executed with `pytest`, exercises five domains: the health endpoint; the authentication flow including login, refresh, and unauthorised access; the products Create-Read-Update-Delete (CRUD) contract; the notification list and unread-count endpoints; and unit tests for scraping primitives such as the price parser, the currency detector, and the GetYourGuide JSON-LD extractor. Tests run against real PostgreSQL and Redis service containers in continuous integration rather than against mock objects, on the principle that mocked persistence catches neither schema drift nor migration errors.

End-to-end frontend verification was implemented as a Patchright-based browser test that scripts a full user journey—authentication, product addition, detail navigation, scrape triggering, notifications, and settings—and asserts on rendered text and the absence of JavaScript exceptions. The script captures screenshots after each step as evidence and was re-run after each major refactor; it surfaced two issues that unit tests would not have caught, namely a numeric formatting bug arising from PostgreSQL’s serialisation of `NUMERIC(12,2)` columns as strings and a 307-redirect-induced header loss caused by FastAPI’s default trailing-slash policy.

The continuous integration pipeline defined in `.github/workflows/ci.yml` is composed of four jobs: a `ruff`-based lint and format check on the backend; the backend test suite with PostgreSQL and Redis as service containers; `npm ci` followed by a Vite production build for the frontend; and, conditional on a push to `main`, a build-and-push of both container images to GitHub Container Registry tagged with both `latest` and the short commit SHA.

7 Discussion

Several engineering decisions warrant comment beyond their description in earlier sections.

The choice of FastAPI over Flask was motivated by the centrality of asynchronous I/O to the workload. Most of the API’s time is spent waiting on either PostgreSQL or Redis; the `async` event loop lets a single Uvicorn worker handle hundreds of concurrent requests without thread pool tuning. Flask’s synchronous Web Server Gateway Interface (WSGI) model would have required either a thread-pooled deployment or the addition of an asynchronous shim such as Quart, both of which would have produced strictly more configuration than FastAPI imposes.

The decision to maintain platform-specific extractors alongside a generic fallback was informed by an empirical observation. Generic heuristics work well on roughly seventy percent of vendors, but the remaining thirty percent embed prices in vendor-specific structures (such as Hepsiburada’s `data-test-id="price-current-price"` attribute) that require targeted logic. The chosen layered

design allows the platform to be extended with new vendors by adding a single extractor class, without modifying existing code paths.

The replacement of vanilla Playwright with Patchright is the most consequential anti-bot decision. Initial scrapes against GetYourGuide returned HTTP 403 from Cloudflare; the same URL fetched through Patchright with a BrowserForge fingerprint returned the rendered page unmolested. The same code path subsequently succeeded against Hepsiburada, which uses a distinct anti-bot vendor. The patch fixes a single Chrome DevTools Protocol command (`Runtime.enable`) whose unmodified emission has, since 2023, been the canonical signal used by major detection vendors.

8 Conclusion

PricePulse demonstrates a complete, twelve-factor-compliant SaaS implementation comprising an asynchronous Python backend, a JavaScript-aware stealth scraping engine, a typed React frontend, an automated testing and image-publishing pipeline, and a deployment posture that is portable across the major managed cloud platforms without code modification. Each technology in the stack was selected to satisfy a concrete requirement rather than as a default. The deliverable repository, the demo recording, and this report together constitute the full submission for the cloud computing course project.

References

- [1] Wiggins, A. *The Twelve-Factor App*. <https://12factor.net/> (accessed 18 May 2026).
- [2] Anthropic-FastAPI documentation. *FastAPI: Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com/>.
- [3] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>.
- [4] Patchright Project. *Patchright: Undetected Playwright Automation*. <https://github.com/Kaliiiiiiiiii-Vinyzu/patchright-python>.
- [5] BrowserForge Project. *BrowserForge: Realistic Browser Fingerprint Generation*. <https://github.com/daijro/browserforge>.
- [6] Sams, S. *ARQ: Async Redis Queue*. <https://github.com/python-arq/arq>.

Table 1: Twelve-factor methodology conformance.

Factor	Implementation in PricePulse
1. Codebase	Single Git repository tracked at <code>sarperfiles/pricepulse</code> ; multiple deploys (local, test virtual private server) target the same source tree.
2. Dependencies	Backend dependencies declared in <code>pyproject.toml</code> via <code>hatchling</code> ; frontend dependencies declared in <code>package.json</code> . No system packages are assumed at runtime beyond what the container image installs.
3. Config	All deploy-specific configuration (<code>DATABASE_URL</code> , <code>REDIS_URL</code> , <code>SECRET_KEY</code>) is read from environment variables via Pydantic settings classes; no values are hard-coded.
4. Backing services	PostgreSQL and Redis are attached resources referenced by URL only. Replacing the local PostgreSQL container with Amazon Relational Database Service (RDS) requires no code change.
5. Build, release, run	The GitHub Actions pipeline produces tagged Open Container Initiative (OCI) images on every merge to the <code>main</code> branch. Releases are immutable image tags (<code>:sha-xxxxxxx</code>); runtime configuration is injected at container start.
6. Processes	Both the API and worker run as stateless processes. All session and cache state lives in PostgreSQL or Redis. Either process can be killed and restarted with no loss of user-visible state.
7. Port binding	The API exports its service by binding directly to a Hypertext Transfer Protocol (HTTP) port via Uvicorn; no external web server is required.
8. Concurrency	The worker scales horizontally by adding replicas that share the Redis job queue. The API scales horizontally behind any standard layer-7 load balancer because every request is stateless.
9. Disposability	Both processes start in under five seconds and shut down gracefully on <code>SIGTERM</code> . The Async Redis Queue (ARQ) worker drains in-flight jobs before exit.
10. Dev/prod parity	Local development uses the same PostgreSQL 16, Redis 7, and Python 3.12 versions as the deployment target. Docker Compose and a production cluster differ in scale, not in components.
11. Logs	All logs are written to standard output as unbuffered streams. Aggregation is the responsibility of the execution environment (Docker, Amazon CloudWatch Logs, or the Elasticsearch–Logstash–Kibana (ELK) stack).
12. Admin processes	One-off tasks such as database migrations (<code>alembicupgradehead</code>) and admin user seeding run inside the same container image as the API, ensuring identical code and dependency contexts.

Table 2: Persistence schema overview.

Table	Purpose	Hot index
users	Account records, JWT subjects, and API keys.	(email)
products	Tracked URLs with platform tag and scrape interval.	(user_id, is_active)
price_history	Time-series of every scrape outcome.	(product_id, scraped_at DESC)
alert_rules	Per-product trigger rules.	(product_id)
notifications	In-application alert feed.	(user_id, is_read, created_at)
scrape_jobs	Worker execution audit.	(status, started_at)

Table 3: Technology stack and assignment-specification mapping.

Layer	Choice and Rationale	Spec example
Language	Python 3.12 with FastAPI; native async concurrency.	Python (Flask)
Database	PostgreSQL 16; relational integrity for user-product-history graph and native INTERVAL type.	AWS RDS
Deployment	Docker Compose locally; OCI images portable to ECS, Cloud Run, or Kubernetes.	Kubernetes / ECS
CI/CD	GitHub Actions; lint, test, build, push to GitHub Container Registry.	GitHub Actions
Caching	Redis 7; dual duty as cache and ARQ broker.	Redis (ElastiCache)
Monitoring	Standard output logs ingested by Docker, CloudWatch, or Datadog.	Prometheus / CloudWatch
Logging	Python logging to stdout; format compatible with ELK and CloudWatch Logs.	ELK / CloudWatch Logs

Table 4: End-to-end scrape verification.

Vendor	Scraper	Price	Currency
Amazon	httpx + BS4	951.26	USD
GetYourGuide	Patchright	69.00	EUR
Hepsiburada	Patchright	17,023.91	TRY
eBay	httpx + BS4	24.98	USD